Azentio.ai Backend: Complete End-to-End Flow

Project: Azentio.ai Intelligent Collaboration Portal

Backend: FastAPI, Python, SQLAlchemy, MySQL 8.0, asyncmy, Alembic, Pydantic v2

Base URL: `http://127.0.0.1:8000`

API Prefix: `/api/v1`

Swagger UI: `http://127.0.0.1:8000/docs`

ReDoc: `http://127.0.0.1:8000/redoc`

OpenAPI: `http://127.0.0.1:8000/openapi.json`

---

1. Purpose of the Platform

The platform connects client organisations, moderators, product owners, and product teams while protecting sensitive observations.

The main flow is:

1. A client organisation submits a private observation.
2. An authorised client user explicitly grants consent.
3. A moderator reviews the observation and approves or rejects it.
4. An approval contains redacted title and redacted content.
5. A moderator publishes the approved redacted learning.
6. Product entitlement determines which organisations can retrieve it.
7. Product owners connect evidence to product improvements.
8. Withdrawal of consent immediately unpublished the related shared-learning entry.
9. Security-sensitive actions are recorded in the audit log.

The original observation content is not used as public shared-learning content. Shared clients receive only the moderator-provided redacted title and content.

---

2. Security and Authentication Model

Authentication

Protected endpoints use HTTP Bearer tokens:

```
Authorization: Bearer <access-token>
```

Access tokens contain the user UUID and token type. Refresh tokens are used only by the refresh endpoint.

The backend:

• Hashes passwords with Argon2.

• Signs tokens using the configured JWT secret and algorithm.

• Rechecks the user account and active status on protected requests.

• Rejects invalid, expired, wrong-type, or inactive-user tokens.

Roles

Roles are loaded from active **`organisation_memberships`** records on the server. They are not trusted from the JWT or frontend request.

Role	Typical responsibility
`client`	Create observations and manage consent for its organisation
`moderator`	Review observations, approve/reject/redact, publish/unpublish
`product_owner`	Create and track product improvements
`employee`	Staff access such as demo enquiries and permitted product administratio
`admin`	Administrative operations, including product entitlements

Tenant isolation

Observations are always filtered by the user's active client organisation memberships. A frontend-provided organisation UUID is not trusted without a server-side membership check.

Private cross-organisation retrieval intentionally returns:

```
{
  "success": false,
  "error": {
    "code": "OBSERVATION_NOT_FOUND",
    "details": []
  }
}
```

This hides resource existence by returning the same 404-style result used for a non-visible observation.

---

3. Application Startup and Runtime

The application entry point is:

```
app.main:app
```

Start locally on Windows:

```
cd D:\All_projects\Project_Azentio_challenge
.\.venv\Scripts\Activate.ps1
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Database settings and the JWT secret are loaded from **`.env`**. The configured application database is MySQL 8.0 using:

```
mysql+asyncmy://...
```

The health endpoint:

```
GET /api/v1/health
```

Expected result:

```
{
  "success": true,
  "message": "Service is healthy",
  "data": {
    "status": "ok"
  },
  "meta": {}
}
```

---

4. Complete API Inventory

There are **21 route paths and 29 HTTP operations**.

Health

Method	Path	Authentication	Purpose
GET	`/api/v1/health`	No	Health status

Authentication

Method	Path	Authentication	Purpose
POST	`/api/v1/auth/login`	No	Login and obtain tokens
POST	`/api/v1/auth/refresh`	No	Rotate access and refresh tokens
GET	`/api/v1/auth/me`	Required	Current user and memberships

Products

Method	Path	Authentication/Role	Purpose
GET	`/api/v1/products`	No	Published product list
GET	`/api/v1/products/{product_id}`	No	Published product detail
POST	`/api/v1/products`	Admin or employee	Create product
PATCH	`/api/v1/products/{product_id}`	Admin or employee	Update product
DELETE	`/api/v1/products/{product_id}`	Admin or employee	Deactivate product

Demo enquiries

Method	Path	Authentication/Role	Purpose
POST	`/api/v1/demo-enquiries`	No	Submit enquiry
GET	`/api/v1/demo-enquiries`	Admin, employee, or moderator	List enquiries
GET	`/api/v1/demo-enquiries/{enquiry_id}`	Admin, employee, or moderator	Get enquiry
PATCH	`/api/v1/demo-enquiries/{enquiry_id}/status`	Admin, employee, or moderator	Change enquiry status

Observations and consent

Method	Path	Authentication/Role	Purpose
POST	`/api/v1/observations`	Active client	Create private observation
GET	`/api/v1/observations`	Active client	List own organisation observations
GET	`/api/v1/observations/{observation_id}`	Active client	Get own observation
POST	`/api/v1/observations/{observation_id}/consent`	Active client	Grant explicit consent
DELETE	`/api/v1/observations/{observation_id}/consent`	Active client	Withdraw consent

Moderation and publication

Method	Path	Authentication/Role	Purpose
GET	**/api/v1/moderation/observations**	Moderator or admin	Moderation queue
POST	**/api/v1/moderation/observations/{observation_id}/decision**	Moderator or admin	Approve/reject and redact
POST	**/api/v1/moderation/observations/{observation_id}/publish**	Moderator or admin	Publish approved learning
DELETE	**/api/v1/shared-learning/{entry_id}**	Moderator or admin	Manually unpublish learning

Entitlements and shared learning

Method	Path	Authentication/Role	Purpose
POST	**/api/v1/entitlements/product**	Admin	Create/update entitlement
GET	**/api/v1/shared-learning**	Active client	List entitled published learning
GET	**/api/v1/shared-learning/{entry_id}**	Active client	Get one entitled published entry

Improvements and audit

Method	Path	Authentication/Role	Purpose
POST	**/api/v1/product-improvements**	Admin or product owner	Create improvement
GET	**/api/v1/product-improvements**	Authenticated, visibility filtered	List improvements
PATCH	**/api/v1/product-improvements/{improvement_id}/status**	Admin or product owner	Transition improvement status
GET	**/api/v1/audit-events**	Moderator or admin	Read audit history

---

5. Workflow A: Authentication

Step 1: Login

```
POST /api/v1/auth/login
Content-Type: application/json
```

```
{
  "email": "user@example.com",
  "password": "example-password"
}
```

The response contains an access token and a refresh token.

Step 2: Call a protected endpoint

```
GET /api/v1/auth/me
Authorization: Bearer <access-token>
```

The response contains the user profile and active organisation memberships.

Step 3: Refresh

```
POST /api/v1/auth/refresh
Content-Type: application/json
```

```
{
  "refresh_token": "<refresh-token>"
```

```
}
```

The refresh token is exchanged for new access and refresh tokens. An access token cannot be used as a refresh token, and an expired or invalid token is rejected.

---

6. Workflow B: Product Catalogue

Products are public only when published.

Browse

```
GET /api/v1/products?page=1&page_size=20
GET /api/v1/products/{product_id}
```

No authentication is required to retrieve published catalogue entries.

Create, update, or deactivate

The caller must be an admin or employee with an active membership:

```
POST /api/v1/products
PATCH /api/v1/products/{product_id}
DELETE /api/v1/products/{product_id}
```

Creation example:

```
{
  "name": "Product Name",
  "slug": "product-name",
  "description": "Product description",
  "category": "billing",
  "is_published": true
}
```

Deactivation sets the product to unpublished; it does not delete the database record.

---

7. Workflow C: Demo Enquiries

Public submission

```
POST /api/v1/demo-enquiries
```

No login is required for submission. Staff roles can retrieve and update the enquiry.

The backend validates the email, organisation, product association, requirements, and status transitions. Standard status progression is managed by the service rather than by caller-selected arbitrary transitions.

---

8. Workflow D: Private Observation and Consent

Create a private observation

```
POST /api/v1/observations
Authorization: Bearer <client-access-token>
```

```
{
  "organisation_id": "<client-organisation-id>",
  "title": "Example title",
  "description": "Example private observation description",
  "product_id": "<optional-product-id>",
  "category": "billing"
```

```
}
```

Required role:

- Active **`client`** membership

Server behavior:

- Rejects an organisation UUID unless it belongs to the caller.
- Stores **`status: private`**.
- Stores **`consent_status: not_granted`**.
- Records the creating user and organisation.

The observation remains private until consent is granted and moderation/publication is completed.

Retrieve an own observation

```
GET /api/v1/observations/{observation_id}
GET /api/v1/observations?organisation_id=<organisation-id>
```

The list is always limited to active client organisation memberships.

Cross-organisation test

A client from another organisation receives 404 **`OBSERVATION_NOT_FOUND`**. This avoids exposing whether the private observation exists.

Grant consent

```
POST /api/v1/observations/{observation_id}/consent
Authorization: Bearer <client-access-token>
```

```
{
  "reason": "Approved for anonymised sharing"
}
```

The request body is optional because the endpoint permits no body in addition to a JSON body.

Server behavior:

- Confirms observation ownership.
- Confirms the caller belongs to the organisation.
- Records granted status.
- Records the granting user and timestamp.
- Adds a consent-history event.

Granting consent does not publish anything automatically.

Withdraw consent

```
DELETE /api/v1/observations/{observation_id}/consent
Authorization: Bearer <client-access-token>
```

```
{
  "reason": "Organisation withdrawal requested"
}
```

Server behavior:

• Confirms active granted consent.

• Records withdrawn status, withdrawal actor, and timestamp.

• Adds a withdrawal consent event.

• Unpublishes all associated shared-learning entries in the same transaction.

• Adds both withdrawal and unpublication audit events.

After withdrawal, the shared entry is no longer returned by the actual shared-learning endpoints.

---

9. Workflow E: Moderation, Redaction, and Publication

1. Moderator opens the queue

```
GET /api/v1/moderation/observations?page=1&page_size=20
Authorization: Bearer <moderator-access-token>
```

Only consent-granted observations without existing moderation decisions appear in the queue.

Required role:

• **`moderator`** or **`admin`**

2. Approve with redaction

```
POST /api/v1/moderation/observations/{observation_id}/decision
Authorization: Bearer <moderator-access-token>
```

```
{
  "decision": "approved",
  "reason": "Safe after organisation-specific identifiers are removed",
  "redacted_title": "Payment calculation issue",
  "redacted_content": "A calculation may differ from the approved amount."
}
```

Approval requires a redacted title and redacted content. Rejection may omit redacted fields.

The system:

• Prevents duplicate moderation decisions.

• Requires current granted consent.

• Stores the reviewer, decision, reason, and redaction.

3. Publish the approved observation

```
POST /api/v1/moderation/observations/{observation_id}/publish
Authorization: Bearer <moderator-access-token>
```

Publication is allowed only if:

• The source observation still has granted consent.

• The moderation decision is approved.

• Redacted title and content are present.

The backend creates one shared-learning entry from the redacted fields. Original observation text is not exposed by the shared-learning response.

4. Manual unpublication

```
DELETE /api/v1/shared-learning/{entry_id}?reason=Moderator+removal
Authorization: Bearer <moderator-access-token>
```

This sets the entry to unpublished and creates an audit event.

---

10. Workflow F: Product Entitlements and Shared Learning

Grant access to an organisation

```
POST /api/v1/entitlements/product
Authorization: Bearer <admin-access-token>
```

```
{
  "organisation_id": "<organisation-id>",
  "product_id": "<product-id>",
  "is_active": true
}
```

Required role:

- **`admin`**

This creates or updates the unique organisation/product entitlement.

List entitled learning

```
GET /api/v1/shared-learning?product_id=<product-id>&page=1&page_size=20
Authorization: Bearer <client-access-token>
```

Only active client organisations can retrieve shared learning.

An entry is visible when:

- It is published.
- The source observation still has granted consent.
- It has no product association, or the organisation has an active entitlement for that product.

Retrieve one entry

```
GET /api/v1/shared-learning/{entry_id}
Authorization: Bearer <client-access-token>
```

Non-entitled, withdrawn, unpublished, or missing entries return 404 **`SHARED_LEARNING_NOT_FOUND`**.

---

11. Workflow G: Product Improvements and Evidence

Create an improvement

```
POST /api/v1/product-improvements
Authorization: Bearer <product-owner-access-token>
```

```
{
  "title": "Improve calculation",
  "description": "Update the calculation based on verified shared evidence.",
  "product_id": "<product-id>",
  "organisation_id": "<optional-client-organisation-id>",
  "evidence_entry_ids": [
    "<shared-learning-entry-id>"
  ]
```

```
}
```

Required role:

- **`admin`**, or
- **`product_owner`**

Evidence is accepted only when:

- The entry is published.
- It belongs to the selected product.
- The source observation currently has granted consent.

Duplicates are prevented by a unique improvement/evidence relationship.

Update improvement status

```
PATCH /api/v1/product-improvements/{improvement_id}/status
Authorization: Bearer <product-owner-access-token>
```

```
{
  "status": "under_review",
  "reason": "Review started"
}
```

Supported status progression:

Current	Allowed next status
proposed	under_review or rejected
under_review	planned or rejected
planned	in_progress or rejected
in_progress	completed or rejected
completed	None
rejected	None

The transition is validated by the server. Invalid transitions return 409 **`INVALID_IMPROVEMENT_TRANSITION`**.

Evidence visibility

When an improvement is listed, evidence can be marked **`active: true`** or **`active: false`**.

Evidence becomes inactive when:

- The shared-learning entry is unpublished.
- Consent on the source observation is withdrawn.

Withdrawn evidence remains linked for audit/history but is no longer treated as active supporting evidence.

---

12. Workflow H: Audit Logging

```
GET /api/v1/audit-events?page=1&page_size=20
Authorization: Bearer <moderator-or-admin-access-token>
```

The audit API is read-only:

- No public create endpoint.

• No public update endpoint.

• No public delete endpoint.

Recorded events include observation creation, consent grant and withdrawal, moderation decisions, publication, unpublication, entitlement changes, improvement creation/status changes, and related security-sensitive operations.

Audit responses intentionally exclude:

• Passwords.

• Access tokens.

• Refresh tokens.

• JWT secrets.

• Original confidential observation content.

Metadata is limited to safe operational fields such as decision, reason, status, product UUID, organisation UUID, and evidence count.

---

13. Database Model

The implementation uses SQLAlchemy models for:

• **`users`**

• **`organisations`**

• **`organisation_memberships`**

• **`products`**

• **`demo_enquiries`**

• **`observations`**

• **`observation_consent_events`**

• **`moderation_reviews`**

• **`shared_learning_entries`**

• **`organisation_product_entitlements`**

• **`product_improvements`**

• **`product_improvement_evidence`**

• **`audit_events`**

Alembic migrations preserve existing data and create the tables required by each phase. The current configured Alembic head is:

```
phase4_platform_workflows
```

`alembic check` reported no model-to-migration differences.

---

14. Validation and Common Errors

All validation failures are returned with HTTP 422 and code **`VALIDATION_ERROR`**.

Common errors:

Status	Code	Meaning
401	**AUTHENTICATION_REQUIRED**	Missing, invalid, or expired access token
401	**INVALID_CREDENTIALS**	Login failed
401	**INVALID_TOKEN**	Invalid refresh token
403	**FORBIDDEN**	Authenticated user lacks role or organisation permission
404	**PRODUCT_NOT_FOUND**	Product does not exist or is not visible
404	**OBSERVATION_NOT_FOUND**	Observation missing or not visible to caller
404	**SHARED_LEARNING_NOT_FOUND**	Entry missing, unpublished, or inaccessible
409	**MODERATION_DECISION_EXISTS**	Observation already reviewed
409	**PUBLICATION_ALREADY_EXISTS**	Shared entry already exists
409	**CONSENT_NOT_ACTIVE**	Consent withdrawal is not valid
409	**INELIGIBLE_EVIDENCE**	Evidence is not published or consented
422	**VALIDATION_ERROR**	Invalid request fields

---

15. Verification Performed

Automated verification:

• Exact command: **`.\\.venv\\Scripts\\python.exe -m pytest -q`**

• Result: **49 passed, 0 failed, 0 skipped**

Database verification:

• MySQL Server: **8.0.46**

• Database: **`azentio_db`**

• Current Alembic revision: **`phase4_platform_workflows`**

• Migration model consistency: no new operations detected

Swagger verification:

• Swagger UI returned HTTP 200.

• ReDoc returned HTTP 200.

• OpenAPI returned HTTP 200 with a valid 3.1.0 schema.

• 21 route paths and 29 operations were discovered.

Live workflow verification:

• Authentication and token refresh succeeded.

• Invalid and expired tokens returned 401.

• Private observation isolation succeeded.

• Consent grant, withdrawal, moderation, and publication succeeded.

• Entitled client access succeeded.

• Non-entitled client access returned 404.

• Shared learning disappeared after consent withdrawal.

• Product improvement and evidence linking succeeded.

• Improvement status transition succeeded.

• Audit listing succeeded without sensitive information.

All live workflow data was synthetic and cleaned up after verification. No real client information was used.

---

16. Frontend Integration Guidance

Frontend integration should:

1. Keep access tokens only in secure application storage and use the **`Authorization: Bearer`** header.
2. On 401, request a new access token using the refresh endpoint or send the user to login.
3. Treat 403 as insufficient server-side role permission.
4. Treat 404 private-resource errors as "not found or not accessible."
5. Never use browser-provided organisation IDs as proof of membership.
6. Never display original observation text in shared-learning views.
7. Display withdrawn evidence as inactive rather than deleting audit/history links.
8. Use pagination fields **`page`**, **`page_size`**, **`total`**, and **`total_pages`**.
9. Use the Swagger UI to inspect exact live request and response schemas.

---

17. Main Documentation

• General API documentation: **`docs/api_documentation.md`**

• Phase 4 API documentation: **`docs/phase4_api_documentation.md`**

• Phase 2 and Phase 3 documentation remain in the **`docs`** directory.

• PDFs correspond to these Markdown documents and can be regenerated with:

```
python docs\generate_pdf.py docs\backend_end_to_end_flow.md docs\backend_end_to_end_flow.pdf
```